MARS HABITAT CONTROL

Team Name : FABIONE

Claudia Giuseppini - Fabio Falconi - Giuseppe Saltarelli - Nicholas Montoni

Task

Mars Habitat Control is an **IoT platform** for monitoring and automating environmental conditions inside a simulated Mars habitat.

The system:

- collects **data** from a simulator
- **normalizes and distributes** sensor data through a message broker
- evaluates **automation rules**
- commands **actuators**
- exposes a **web dashboard** for real-time supervision.

Environment

Telemetry sensors:

- greenhouse_temperature
- entrance_humidity
- co2_hall
- hydroponic_ph
- water_tank_level
- corridor_pressure
- air_quality_pm25
- air_quality_voc

REST Sensors:

- solar_array
- radiation
- life_support
- thermal_loop
- power_bus
- power_consumption
- airlock

Actuators:

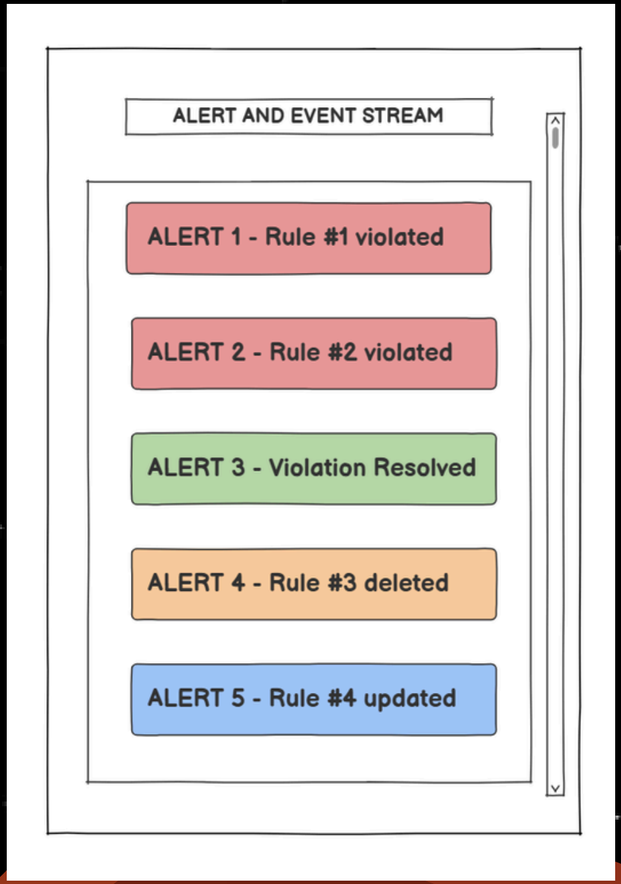
- cooling_fan
- habitat_heater
- entrance_humidifier
- hall_ventilation

User Stories

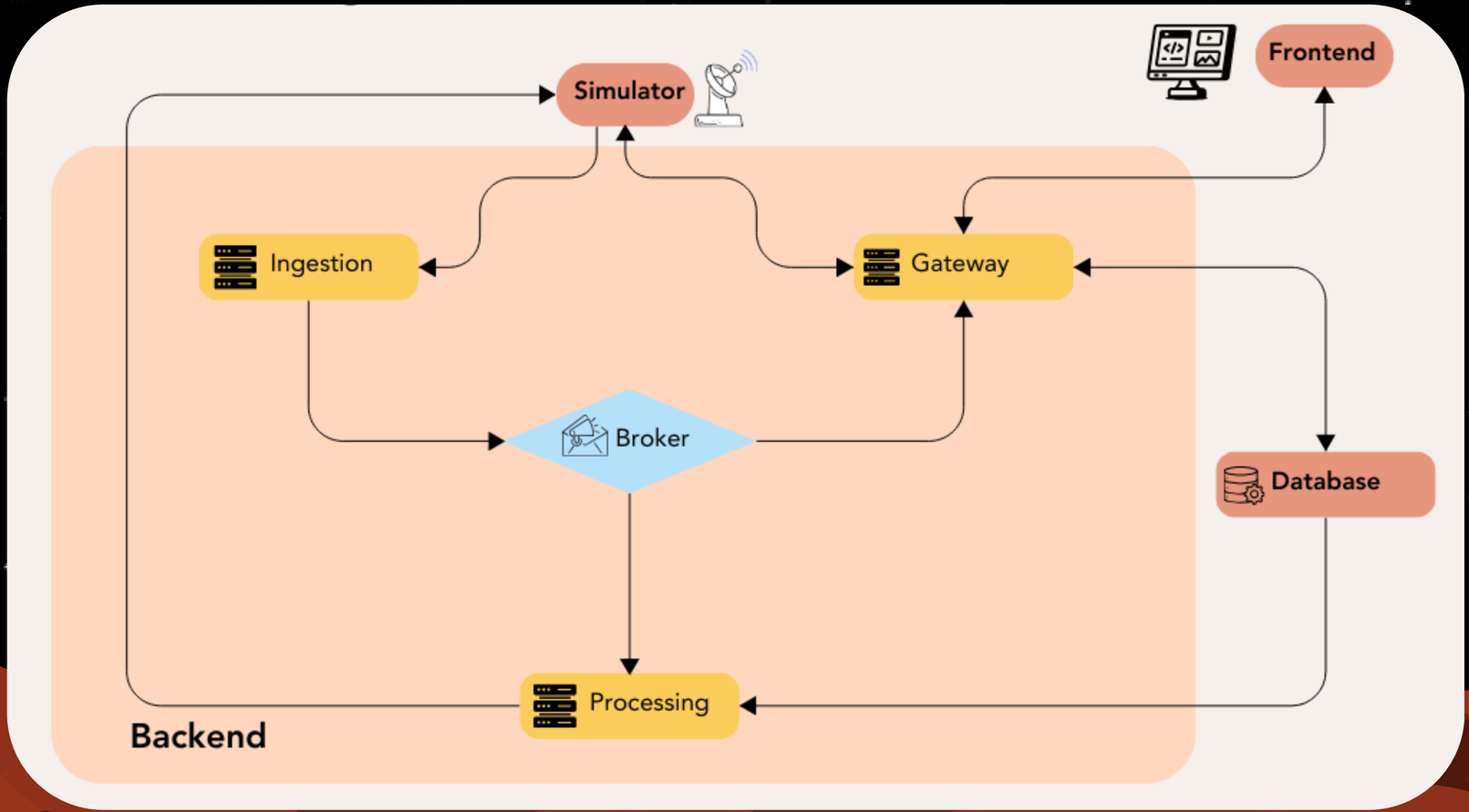
User stories are defined in the User Stories Booklet.

NUMBER 20

As a User, I want to be **notified when a rule has been broken**, a violation has been resolved and when rules have been modified.



Architecture Design



Docker Containers

- The system is deployed using Docker Compose, which orchestrates all services in a **single multi-container** environment.
- Each component **runs in its own container**: simulator, broker, ingestion service, processing service, gateway, database, and frontend.
- Containers communicate through a **dedicated bridge network** (iot-network), ensuring isolated and consistent inter-service communication.
- The configuration also defines port mappings, environment variables, volumes, dependencies, and **health checks**, improving deployment reliability and service monitoring.

Backend

Message Broker

- The system uses a message broker based on a **Publish/Subscribe** (Pub/Sub) architecture to distribute sensor data between services.
- Components publish and subscribe to **topics**, enabling **asynchronous and decoupled** communication.
- The broker is implemented using **ActiveMQ** running in a container, ensuring **portability** and easy deployment.
- It acts as the central hub for normalized sensor data used by monitoring, rule evaluation, and actuator control services.

Ingestion

- The ingestion service collects sensor data from the simulator using **dedicated functions** for both sensor types.
- Retrieved values are **normalized** into a consistent format to ensure uniform processing across the system.
- The normalized data is then **published** to the message broker.
- This allows other services to **subscribe** and process sensor **updates in real time**.

Normalized Schema

```
{
  "sensor_id": "string",        // Unique ID
  "sensor_type": "string",      // "sensor" or "telemetric"
  "timestamp": "ISO8601/UTC",   // Captured/Event time
  "source": "string | null",    // Origin location or subsystem
  "status": "string",           // Current status (e.g., ok, warning, IDLE)
  "metrics": [                  // List of measurements
    {
      "name": "string",         // Metric name
      "value": "number",        // Measured value
      "unit": "string | null"   // Unit of measurement
    }
  ]
}
```

Processing (1)

- The processing service receives sensor data by **subscribing** to the message broker.
- It retrieves the **automation rules** stored in the database.
- Incoming sensor values are **compared** against the defined rules.
- When a rule is triggered, the service **takes a decision** and sends the **corresponding command** to the actuator.

Processing (2)

- The service subscribes to the message broker to receive normalized sensor data and uses the following query to interrogate the database:

```
query = """
    SELECT id, metric_name, operator, threshold, actuator_name, action_value
    FROM automation_rules
    WHERE sensor_name = %s AND enabled = TRUE
"""
```

- It compares sensor metrics with the rule thresholds and applies a **majority voting** mechanism across the metrics to determine whether to trigger an actuator command.

Gateway

- The gateway subscribes to the message broker to **receive normalized** sensor data.
- It **forwards real-time data and current rules** in the database to the frontend dashboard for monitoring the habitat conditions.
- It **manages rule operations** from the frontend (e.g. add, edit, delete) by interacting with the database.
- It handles **actuator controls** from the frontend, sending commands to turn simulation actuators on or off manually.

Database

Database

- The system uses a **relational database** to store and manage automation rules.
- It is implemented using the **MariaDB** container image.
- The database **persists rule** definitions and initialization.
- Services query it to **retrieve, update, or delete rules** used in the automation logic.

Rule Table

```sql
CREATE TABLE IF NOT EXISTS automation_rules (
    id INT AUTO_INCREMENT PRIMARY KEY,
    sensor_name VARCHAR(255) NOT NULL,
    metric_name VARCHAR(255) NOT NULL,
    operator VARCHAR(2) NOT NULL,      -- <, <=, =, >, >=
    threshold FLOAT NOT NULL,
    actuator_name VARCHAR(255) NOT NULL,
    action_value VARCHAR(10) NOT NULL, -- ON / OFF
    enabled BOOLEAN DEFAULT TRUE
);
```

Frontend

Frontend (1)

- The frontend interface is built using **HTML** to **structure the dashboard** and display system information.
- The layout and responsive design are implemented using the **Bootstrap framework**.
- Bootstrap components (such as cards, tables, and buttons) are used to **organize** sensor data, rules, and actuator controls.
- This approach provides a **clean, responsive, and user-friendly interface** for monitoring and managing the Mars Habitat system.

Frontend (2)

- JavaScript fetches real-time data from the gateway API endpoints exposed by the backend.
- Retrieved sensor data is **processed and dynamically inserted** into the HTML elements of the dashboard.
- The script **updates the interface in real time**, reflecting sensor values, rule status, and actuator states.
- It also handles **user interactions** (e.g., add/edit/delete rules and actuator controls) by sending requests to the gateway API.

Thanks for your attention